

Lecture 5 BFS, DFS, Shortest Paths Problem

Breadth First Search

Overview

- Applications of Graph Search
- Graph Representations
- Breadth-First Search

Graph Representations

Adjacency Lists:

for each vertex u , $\text{Adj}[u]$ stores u 's neighbors.

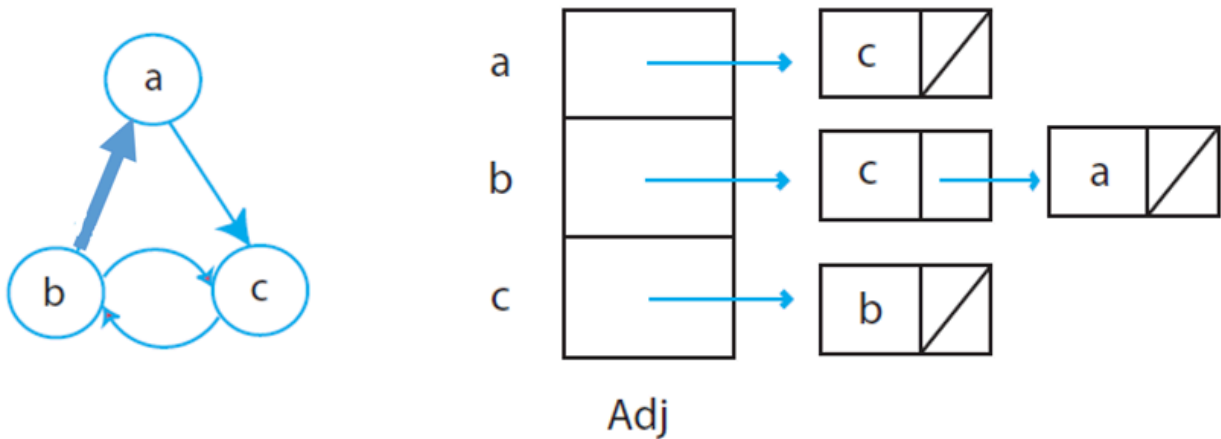


Figure 2: Adjacency List Representation: Space $\Theta(V + E)$

BFS

Explore graph level by level from s , level 0 is $\{s\}$, level i is vertices reachable by path of i edges but not fewer. Build level i from level $i - 1$ by trying all outgoing edges but ignoring vertices

from previous levels.

Breadth-First-Search Algorithm

```
BFS (V,Adj,s):
    level = { s: 0 }
    parent = { s : None }
    i = 1
    frontier = [s]                                # previous level, i - 1
    while frontier:
        next = [ ]                                # next level, i
        for u in frontier:
            for v in Adj[u]:
                if v not in level:                # not yet seen
                    level[v] = i                 # = level[u] + 1
                    parent[v] = u
                    next.append(v)
        frontier = next
        i += 1
```

$O(V + E)$ time. (Linear time)

Shortest Paths

BFS is used to find shortest paths.

For every vertex v , fewest edges to get from s to v is

$$\begin{cases} \text{level}[v] & \text{if } v \text{ assigned level} \\ \infty & \text{else (no path)} \end{cases}$$

parent pointers form shortest-path tree. Take $v, \text{parent}[v], \text{parent}[\text{parent}[v]]$, etc., until s as the shortest path.

Depth First Search

DFS

It will follow path until get stuck. Then backtrack until reach unexplored neighbor. Recursively explore.

```

parent = {s: None}

DFS-visit (V, Adj, s):
  start → for v in Adj [s]:
    v      if v not in parent:
            parent [v] = s
            DFS-visit (V, Adj, v)
  finish → v

DFS (V, Adj)
  parent = { }
  for s in V:
    if s not in parent:
      parent [s] = None
      DFS-visit (V, Adj, s)

```

search from start vertex s (only see stuff reachable from s)

explore entire graph

$O(V + E)$ time. (linear time)

Edge Classification

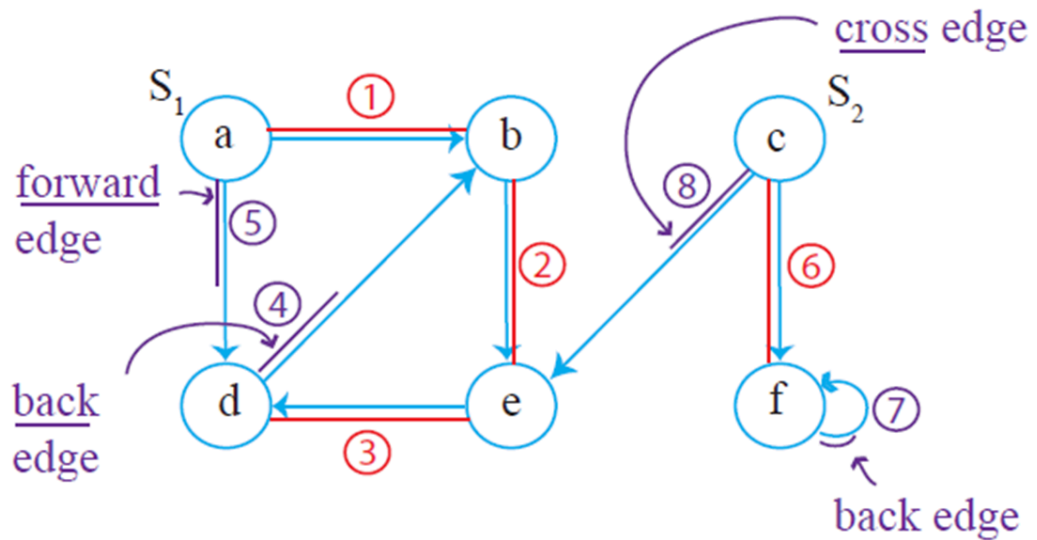
In DFS:

- tree edges: formed by parent
- nontree edges
 - back edge: to ancestor
 - forward edge: to descendant
 - cross edge: to another subtree

To compute this classification(back or not), mark nodes for duration on the stack.

Only tree and back edges in undirected graph.

Example



Cycle Detection

Graph has a cycle $\Leftrightarrow$ DFS has a back edge.

Job Scheduling

Given directed acyclic graph(DAG), where vertices represent tasks and edges represent dependencies, order tasks without violating dependencies.

Use topological sort.

Topological Sort

- DFS for the graph and record finishing time for each vertex(`order[v]`)(time at which `DFS-Visit(v)` finishes)
- reverse order after DFS and get topological sort.

Shortest Path Problem

Formulation: Problem on a weighted graph $G(V, E), W : E \rightarrow R$

Two algorithms: Dijkstra $O(V \log V + E)$ assumes **non-negative edge weights**. Bellman Ford $O(VE)$ for **general algorithm**.

Subpaths of shortest paths are shortest paths

Single Source Shortest Paths

Given $G = (V, E)$, w and a source vertex S , find $\delta(S, V)$ and the best path from S to each $v \in V$.

Negative-Weight Edges

If the graph has **negative cycles**, it will make the certain shortest paths undefined since the path can go around the cycle many times.

Triangle Inequality

For all $u, v, x \in X$, we have

$$\delta(u, v) \leq \delta(u, x) + \delta(x, v)$$

Dijkstra Algorithm

Relax Edge (u, v) :

```
if d[v] > d[u] + w(u, v):  
    d[v] <- d[u] + w(u, v)  
    pre[v] <- u
```

It maintains the invariant that $d[v] \geq \delta(s, v)$ for all $v \in V$.

Shortest Path in DAGs

It have no negative cycles since there are no cycles in the graph. So it allows negative edges.

- Topologically sort DAG to get the linear order.
- One pass over vertices in topologically sorted order relaxing each outgoing edge for each vertex.

$\Theta(V + E)$ time. (topological sort $O(V + E)$ + traverse $O(V + E)$).

procedure explore (G, v)

Input: $G = (V, E)$, $v \in V$

Output: `visited[u]` is set to true for all nodes u reachable from v .

```
visited[v] = true;  
previsit(v)  
for each edge (v,u) in E:  
    if not visited(u): explore(u)  
postvisit(v);
```

```
procedure dfs(G)
```

```
for all v in V  
    visited(v) = false
```

```

for all v in V
    if not visited(v): explore(v);

```

Dijkstra's Algorithm

Maintain a set S of vertices whose final shortest path weights have **been determined**.

Repeatedly select $u \in V$ with minimum shortest path estimate, add u to S and relax all edges of u .

```

Dijkstra (G,W,s) // use priority queue Q
    Initialize(G,s)
    S ← empty set
    Q ← V[G]
    while Q is not empty set
        do u ← extract-min(Q)
        S = S + {u}
        for each vertex v in Adj[u]
            do relax(u,v,w)

```

The running time of Dijkstra's algorithm depends heavily on the priority queue implementation.

Implementation	deletemin	insert/ decreasekey	$ V \times \text{deletemin} + (V + E) \times \text{insert}$
Array	$O(V)$	$O(1)$	$O(V ^2)$
Binary heap	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
d -ary heap	$O(\frac{d \log V }{\log d})$	$O(\frac{\log V }{\log d})$	$O((V \cdot d + E) \frac{\log V }{\log d})$
Fibonacci heap	$O(\log V)$	$O(1)$ (amortized)	$O(V \log V + E)$

Bellman-Ford

Bellman-Ford(G,W,s)

```

Initialize()
for i = 1 to |V|-1
    for each edge (u,v) in E
        Relax(u,v)
for each edge (u,v) in E
    do if d[v] > d[u] + w(u,v)
        then report a negative-weight cycle exists

```

Since at most the path passes $V - 1$ edges from s can reach any vertex, after $V - 1$ times traverse the shortest path for each vertex can be ensured.

At the end, $d[v] = \delta(s, v)$, if no negative-weight cycles.
 $O(VE)$ time.

Speeding up Dijkstra

Single-Source, Single-Target Dijkstra

```
Initialize()
Q <- V[G]
while Q is not empty
  do u <- extract_min(Q) until u = t // stop on the target
  for each vertex v in Adj[u]
    do relax(u, v, w)
```

Bi-Directional Search

Alternate forward search from s and backward search from t in the reversed graph. $d_f(u)$ distances for forward search and $d_b(u)$ distances for backward search. Algorithm terminates when some vertex w deleted from the queue of both searches Q_f and Q_b . After terminating, find node x with minimum value of $d_f(x) + d_b(x)$. Note that x may be not the terminated node.

But nowadays the terminated condition is more accurate.

- Maintain forward search queue `dist_f` and backward search queue `dist_b`.
- Maintain a current shortest length μ (infinity initially). When dequeue a vertex u and relax related edges, check whether another direction has calculated the dist. If $dist_f[u] + dist_b[u] < \mu$, then renew $\mu = dist_f[u] + dist_b[u]$.
- Main loop: each choose the vertex with smaller `dist` in two queues.
- Terminate: Let f_{min}, b_{min} be the smallest $dist_f$ and $dist_b$. If $f_{min} + b_{min} \geq \mu$, then μ is the shortest length and terminate.